

TripMind

AI-powered personalized travel experiences

Group 3



Our team

1. *Trần Nguyễn Mỹ Anh – 20235474*
2. *Nguyễn Thu Huyền – 20235507*





Table of contents

01

Introduction

Context and Motivation
Project goals and The MAS
Our contributions

02

Dataset

Data collection
Data preprocessing
Data analysis



03

Methodology

Architecture of Tripmind

04

Experiments



05

Demo



Introduction



Vietnam Infographic Map



National flag



Population 97,338,056

Language

Vietnamese

Currency

VND





Context



The tourism industry is shifting towards Gen Z and Millennials who prefer personalized, “experience-based” journeys and hidden gems rather than rigid tours

Some challenges

Semantic Gap

**Information
Overload**

**Quantitative
Bias**

**Fragmented
Planning**

**Communication
Barrier**

“Due to these difficulties, we decided to
developed a project called by”

—TripMind—



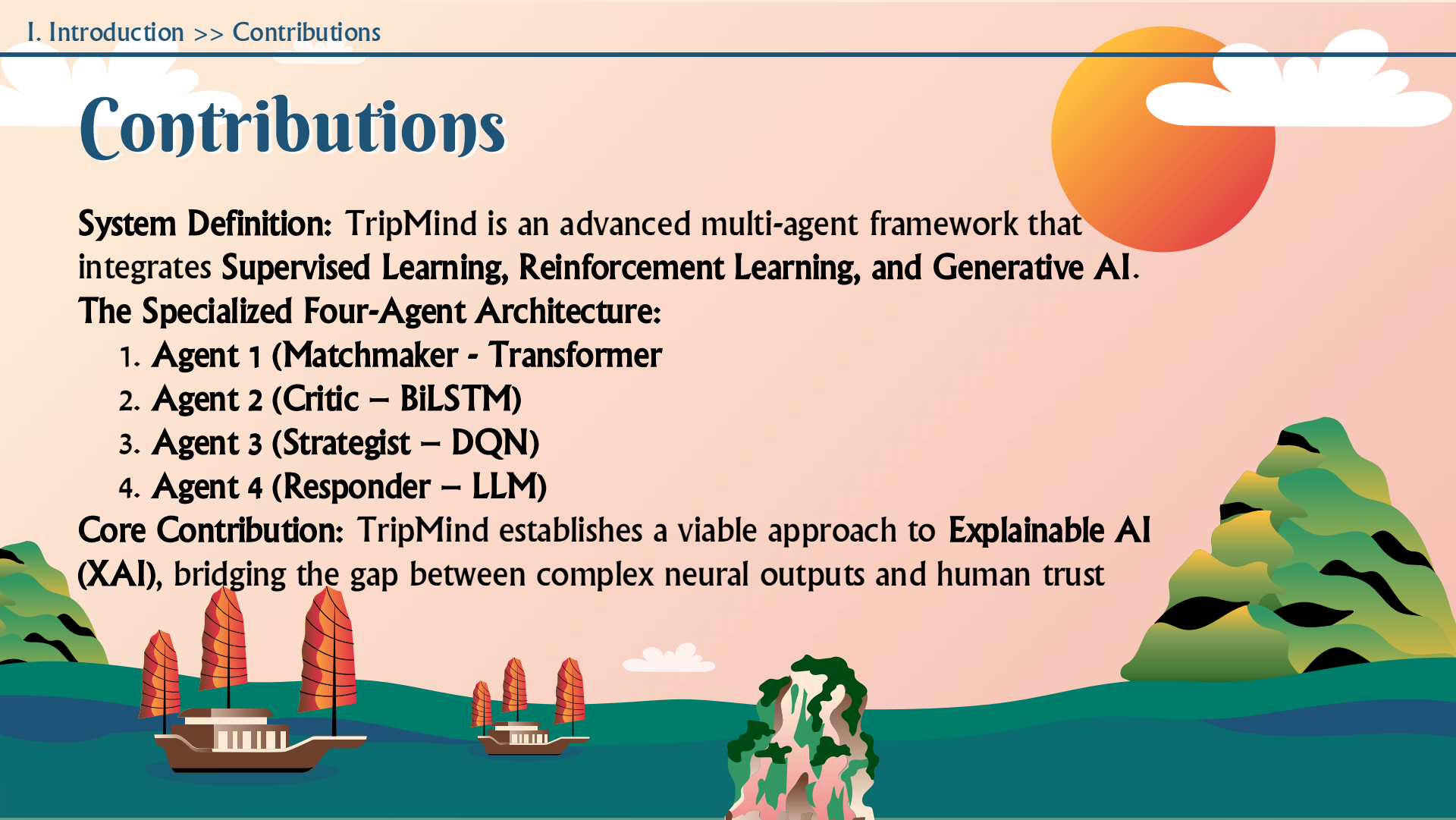
Contributions

System Definition: TripMind is an advanced multi-agent framework that integrates **Supervised Learning, Reinforcement Learning, and Generative AI.**

The Specialized Four-Agent Architecture:

1. **Agent 1 (Matchmaker - Transformer)**
2. **Agent 2 (Critic – BiLSTM)**
3. **Agent 3 (Strategist – DQN)**
4. **Agent 4 (Responder – LLM)**

Core Contribution: TripMind establishes a viable approach to **Explainable AI (XAI)**, bridging the gap between complex neural outputs and human trust



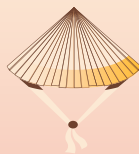


Related Works

Core technologies



Transformer & Self-Attention



**Bidirectional Long Short-Term
Memory (BiLSTM)**



Deep Q-Learning (DQN)

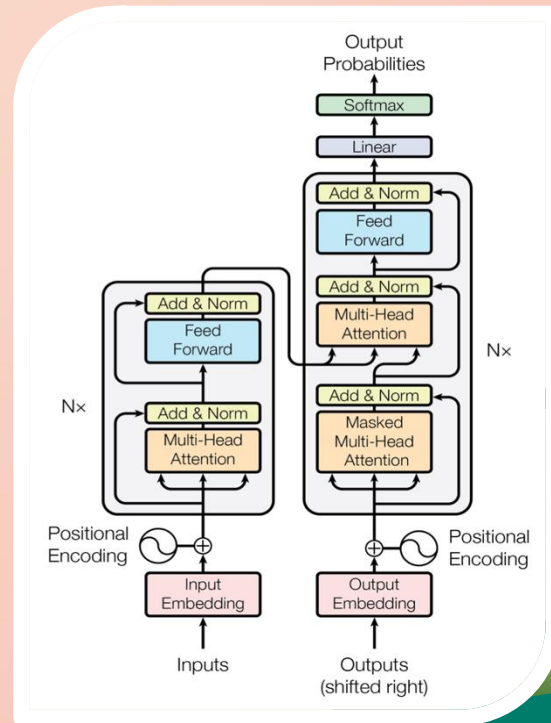


Agent AI



Transformer & Self-Attention

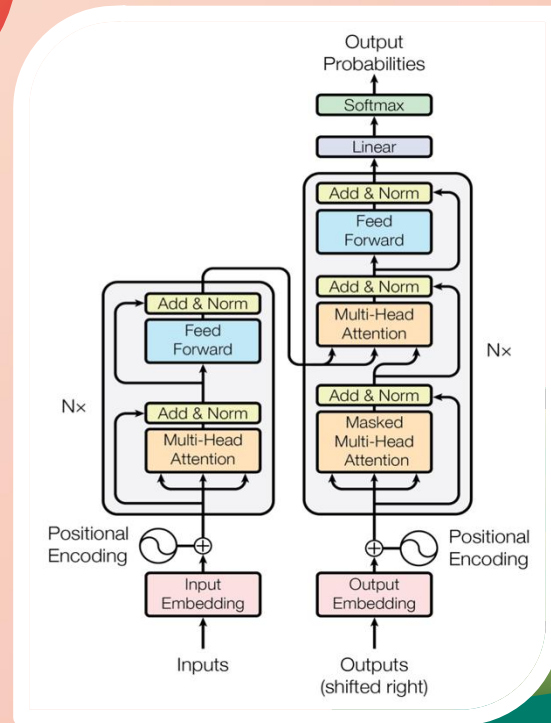
Transformers were developed to replace sequential RNNs/LSTMs to solve slow training and vanishing gradient problems.



Transformer & Self-Attention

The Transformer allows for the **simultaneous** computation of all tokens in a sequence

- Eliminates sequential dependency
- Maximize the power of the GPU
- Significantly shorten training time

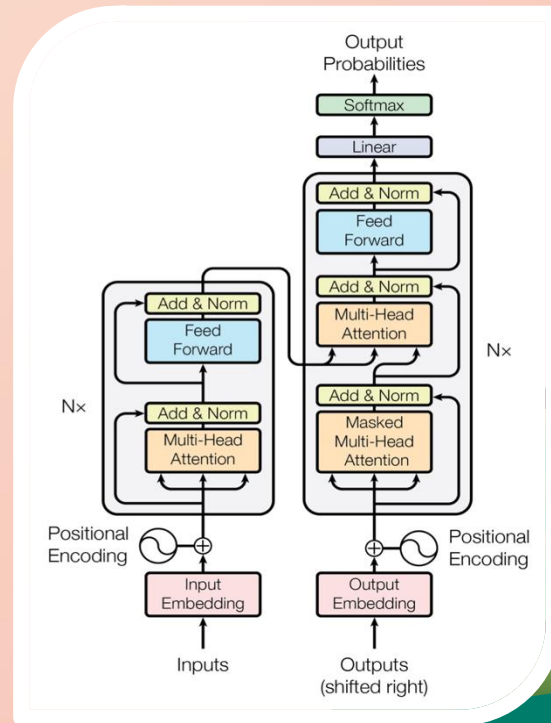


Transformer & Self-Attention

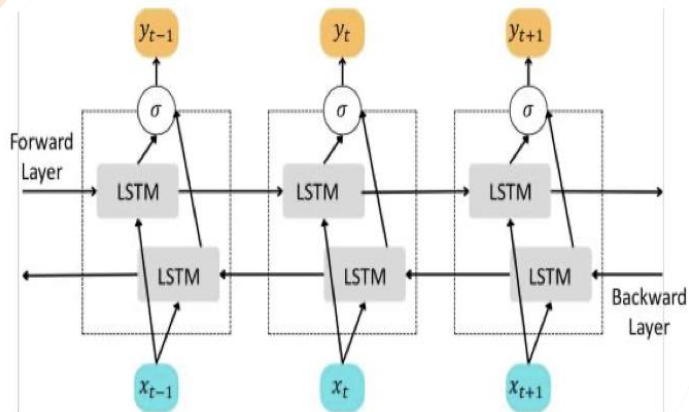
Utilizes **Scaled Dot-Product Attention** to assign weights to words based on semantic correlation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)V$$

- Preserves semantic integrity in long descriptive queries



Bidirectional LSTM



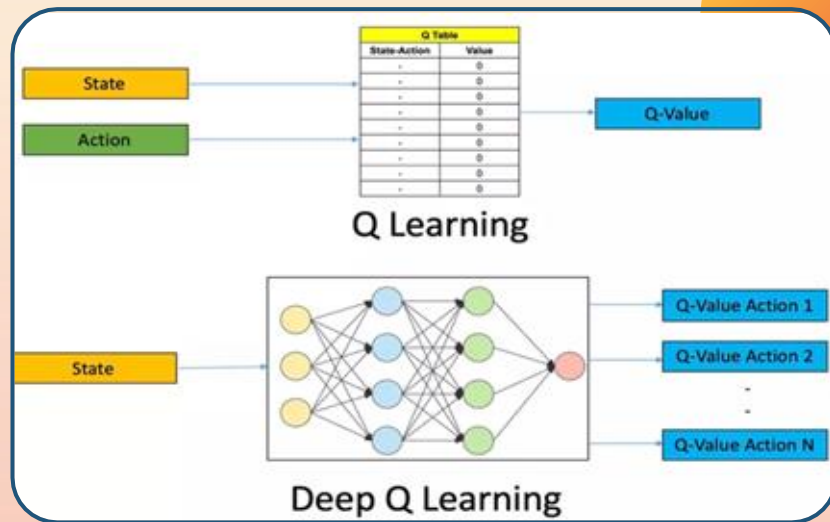
BiLSTM is a variation of LSTMs that processes the sequence of inputs in both the forward and reverse directions

- Allows the model to have a more complete understanding of temporal dependencies

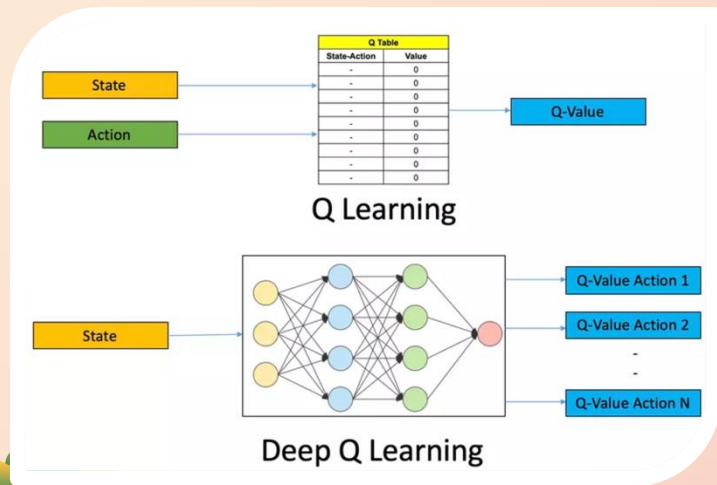


Deep Q-Learning

Evolution: Moves from Q-tables (limited by size) to Neural Networks to approximate optimal actions



Deep Q-Learning



Function: Uses a loss function based on Temporal Difference (TD) Error to learn policies.

$$Loss = E \left[(Target - Q(s, a))^2 \right]$$

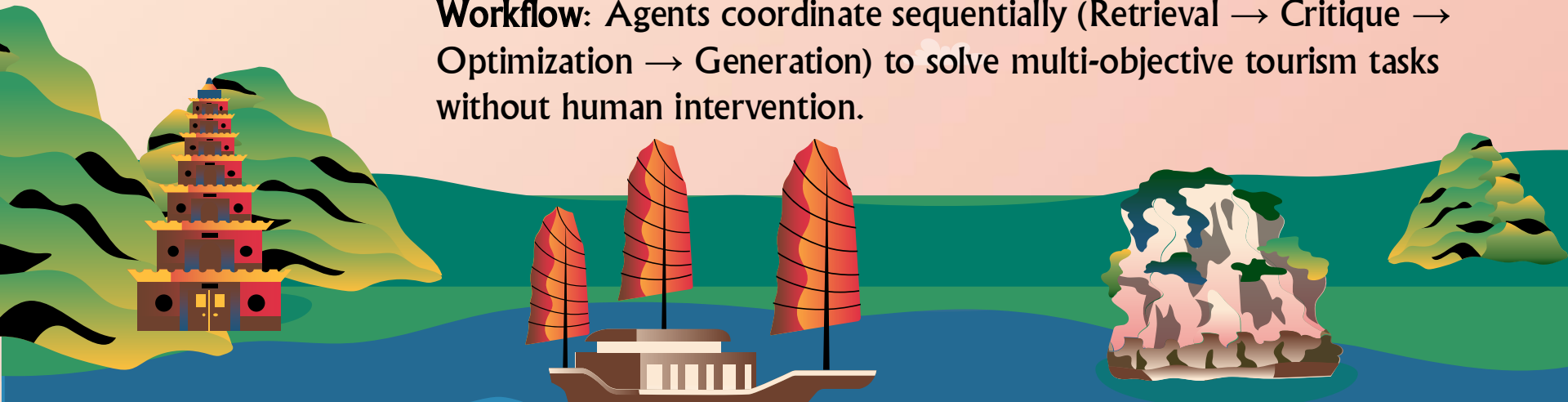
Role: Handles continuous, massive state spaces (GPS coordinates) for solving the Route Optimization (TSP) problem effectively.

Agent A1

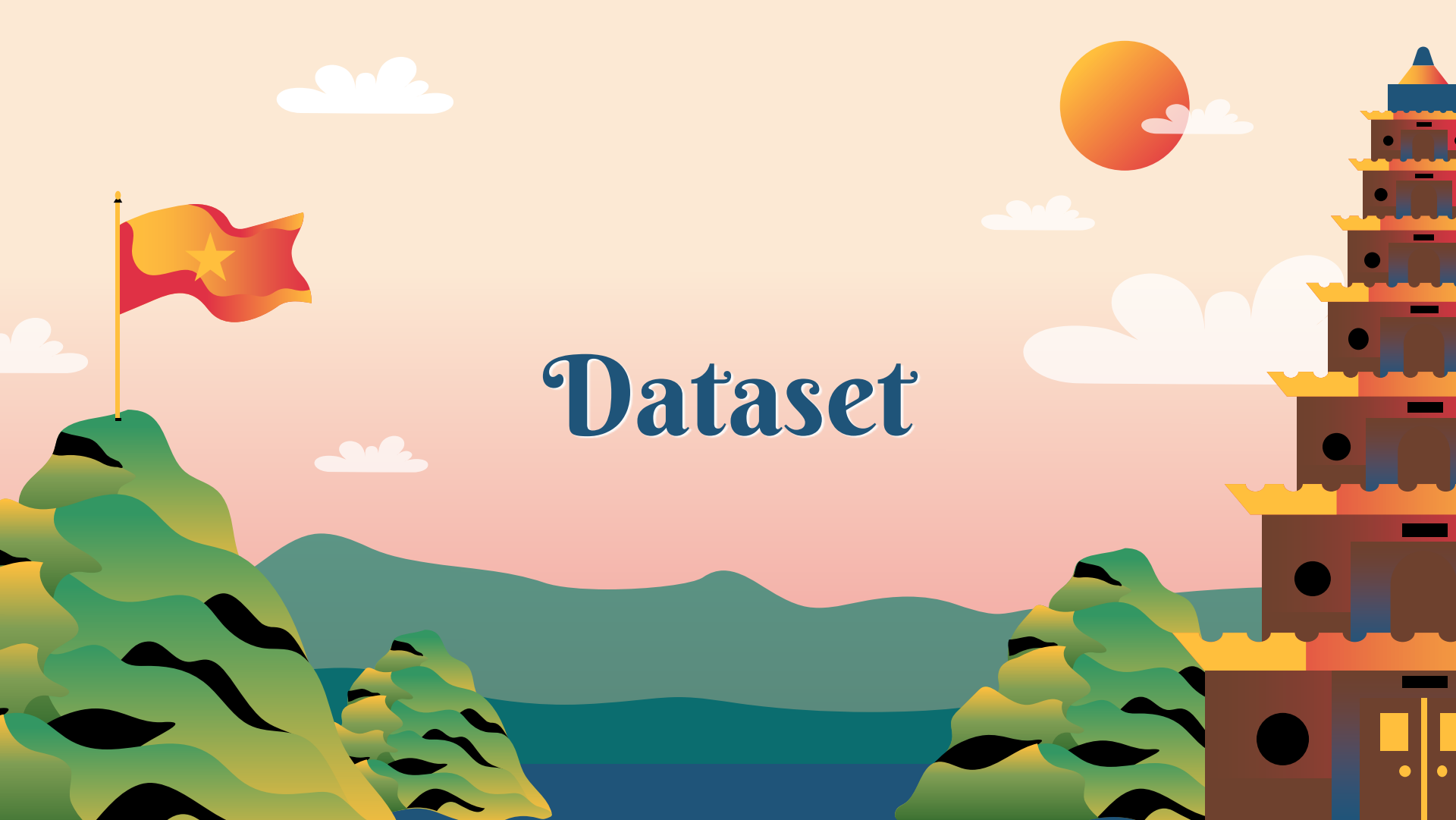
Concept: Transition from monolithic models to autonomous, specialized entities.

Properties: Autonomy, Social Ability, Reactivity, and Proactiveness.

Workflow: Agents coordinate sequentially (Retrieval → Critique → Optimization → Generation) to solve multi-objective tourism tasks without human intervention.



Dataset



Dataset



Data collection



**Data
preprocessing**



Data analysis

Data Sourcing & Strategy

Primary Sources: TripAdvisor (for professional, detailed opinions and specific pros/cons) and Google Maps (for a broad, diverse user base reflecting general consensus)

Scope: Targeted 34 Vietnamese provinces, limiting to a maximum of 50 attractions per province to maintain data quality and minimize noise

Quality Control: Prioritized reviews sorted by "most detailed" to ensure high-quality textual input for sentiment analysis

Harvesting Tools & Pipeline

TripAdvisor

API Integration: Omkar's scraping tool
to extract data
from TripAdvisor



tripadvisor®



Google Maps

Google Maps

Dynamic Scraping: Used the
Playwright library to control
Chromium browser



Data preprocessing

Output: Generated two distinct datasets: **Attractions dataset** (metadata) and **User Reviews dataset** (textual feedback)

For **Attractions dataset**'s field:

- ☐ **Link:** Black Virgin Mountain (Tây Ninh, Việt Nam) - Đánh giá - Tripadvisor
- ☐ **Name:** Black Virgin Mountain
- ☐ **Rating:** 4.1
- ☐ **Reviews:** 81
- ☐ **Categories:** [{ "id": 11089, "name": "Núi" }]
- ☐ **Tripadvisor_entity_id:** 2494718
- ☐ **Province:** Tây Ninh
- ☐ **New_province:** Tây Ninh



Data preprocessing

For User Reviews dataset's Field:

- | | | |
|--------------------------------------|--|--|
| ☐ Text | ☐ like_count | ☐ published_at_date |
| ☐ Trip | ☐ review_tip | ☐ destination_id |
| ☐ Title | ☐ review_link | ☐ province_id |
| ☐ Ratings | ☐ is_translated | ☐ Province |
| ☐ Language | ☐ owner_response | ☐ new_province |
| ☐ Reivewer | ☐ created_at_date | |
| ☐ Reivewer_id | ☐ original_language | |



Data Cleaning & Normalization



Geocoding: Integrated SerpAPI to access Google Maps API for coordinate retrieval and name normalization.

Deduplication: Mapped inconsistent attraction names to a single “Normalized Name” on Google Maps to remove duplicates.

Noise Filtering:

- Removed non-physical entities (e.g., travel agencies, tour operators, cruises).
- Excluded reviews with 0 content, nonsense text, or non-Vietnamese language





Feature Engineering & Selection

Geographical Update: Replaced outdated province names with current administrative divisions by querying Google Maps.

Redundancy Reduction:

- Removed “Review Titles” (content already covered in text).
- Retained only “Published Date” while discarding “Stay Date” due to high correlation.

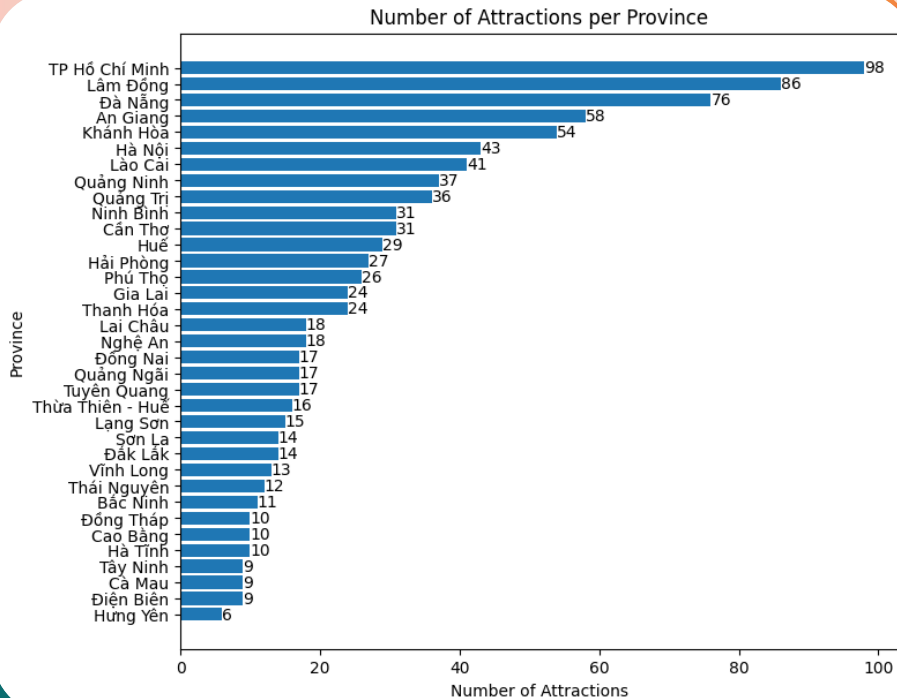
Feature Enrichment: Merged attraction metadata (categories, coordinates) with individual reviews to create a unified analysis-ready dataset

Dataset Statistical Overview

Volume: 14,000+ reviews across 964 attractions.

Coverage: 34 provinces, 122 different categories.

Missing Values: Identified that 52% of reviews lack "Trip Type" information, while coordinates and ratings are 100% complete

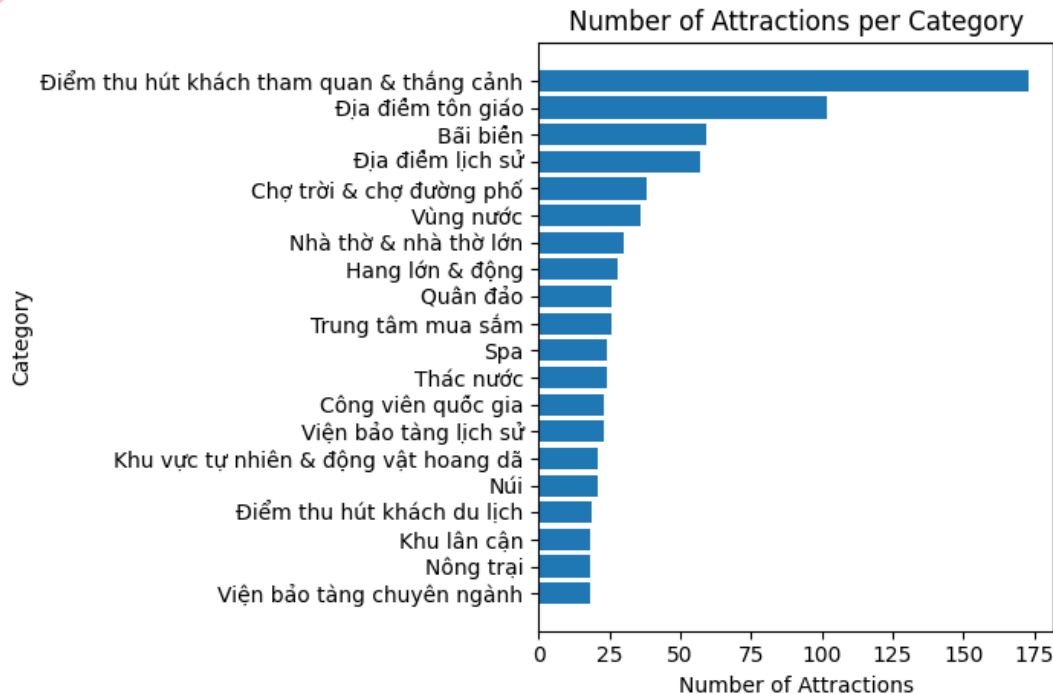


Category Distribution & Long-tail Analysis

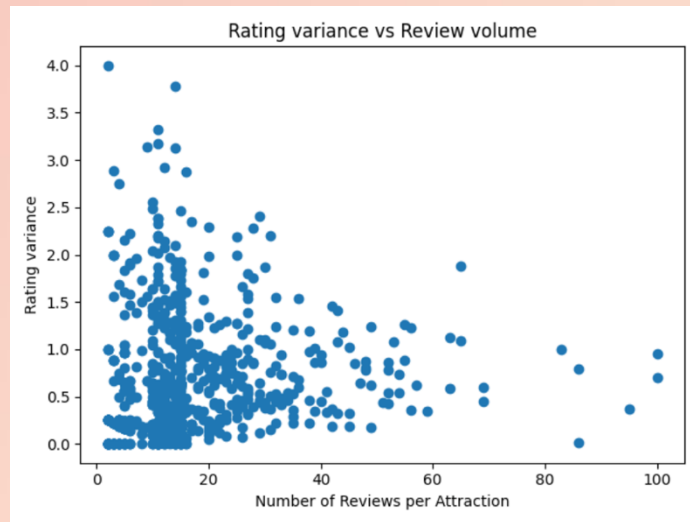
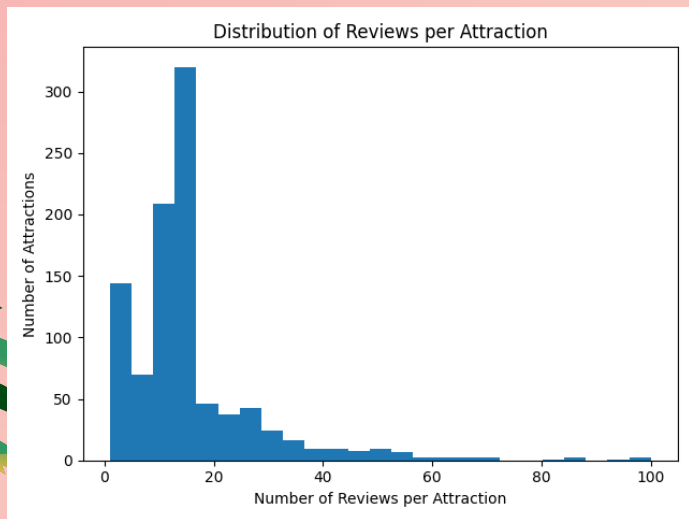
Dominant Categories: “Tourist attractions & scenic spots” (17.93%) is the most frequent.

Distribution: Top 10 categories account for **59.59%** of the data, indicating a highly imbalanced, long-tailed distribution.

Insight: Specialized categories (e.g., Spas, Farms) have significantly fewer samples than broad categories



Review Volume & Reliability



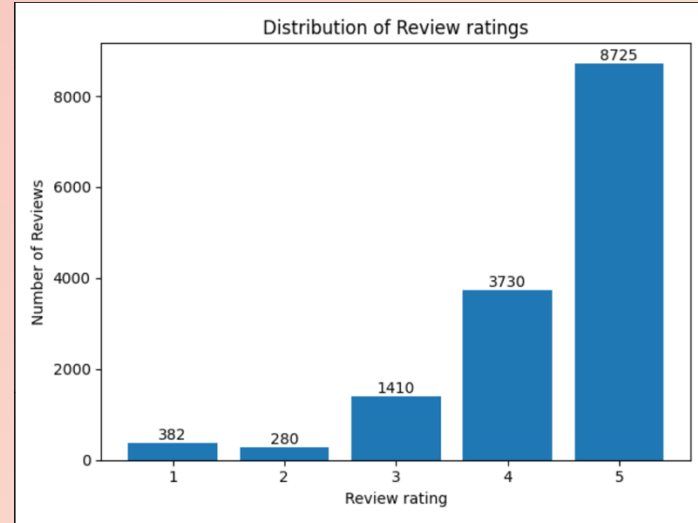
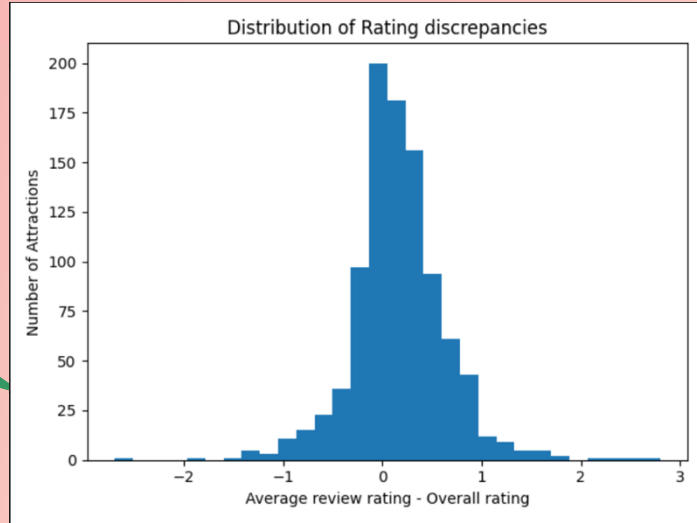
Central Tendency: Median of 14 reviews per attraction (IQR: 10-15).

The Law of Large Numbers: Attractions with <20 reviews show high rating volatility.

Variance stabilizes (mostly <1.5) as volume reaches 80-100 reviews.

Skewness: Most attractions have few reviews, while a small elite group attracts the majority of feedback

Rating Discrepancy & Bias Analysis



Skewed Positivity: 60.2% of all reviews are 5-star; mean rating is 4.39.

Cross-platform Consistency: 73.8% of attractions show <0.5 star discrepancy between platforms, validating overall data faithfulness.

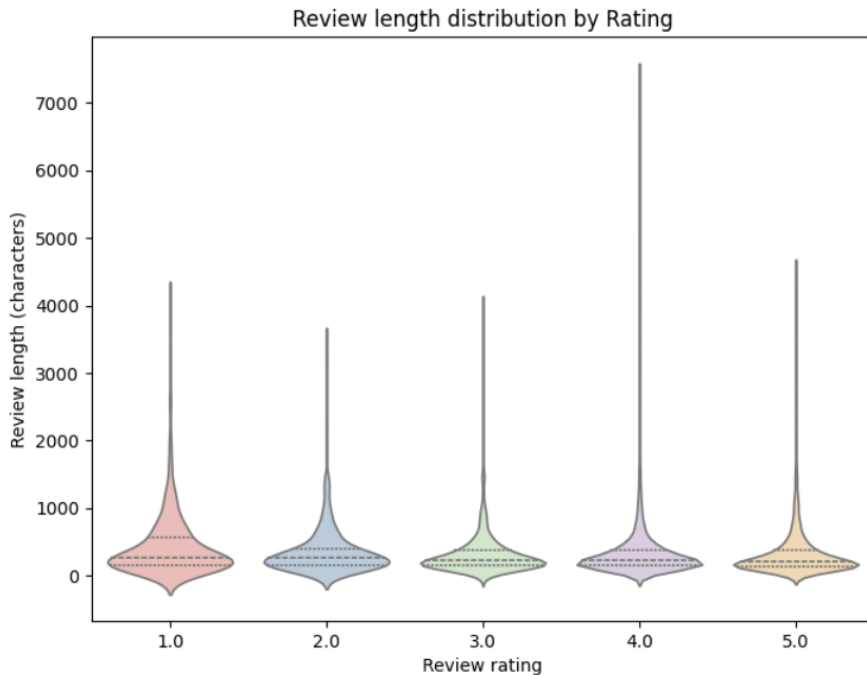
Demographics: Reviews are dominated by "Friends" trips (31.4%) and "Couples" (26.4%), with business trips being the least represented (4.2%)

Linguistic & Textual Analysis

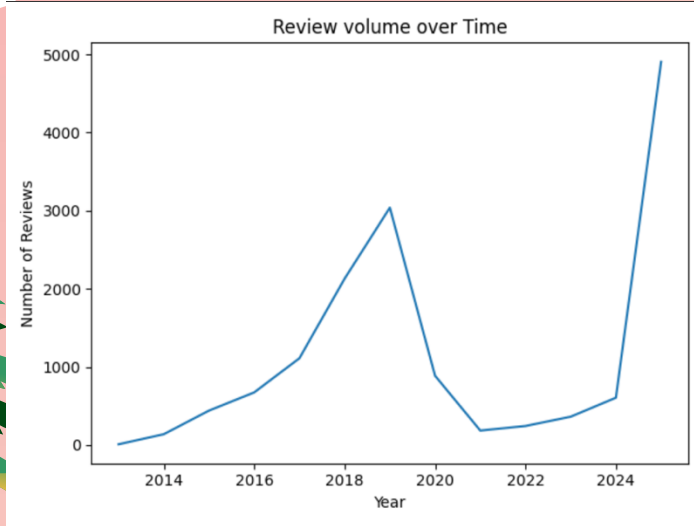
Review Length: Median length is 224 characters. 90% are under 694 characters.

Lexical Diversity (TTR): Higher-rated reviews use more nuanced vocabulary (1-star TTR ≈ 0.798 vs. 5-star TTR ≈ 0.849).

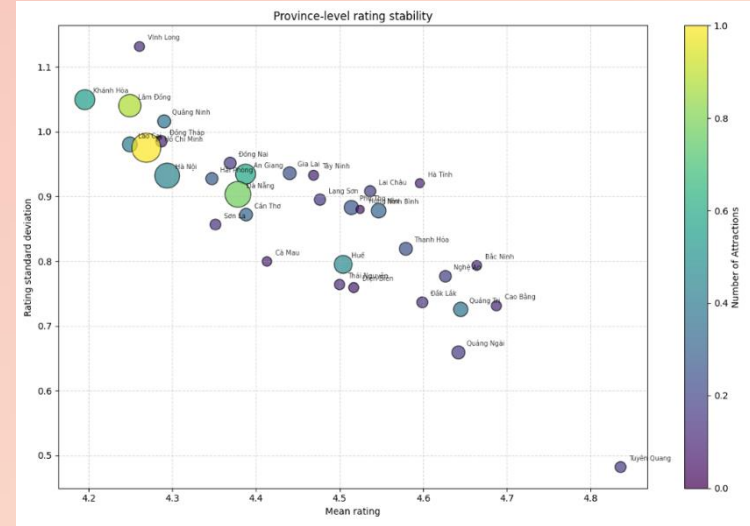
Complexity: Satisfied users (4-5 stars) tend to write significantly longer and more linguistically complex reviews



Spatio-Temporal Insights



Temporal Trend: Steady growth (2013-2019) followed by a **54.4% drop** during COVID-19. A massive **396.8% surge** occurred in 2024-2025.



Geographic Stability: Provinces like Tuyen Quang and Cao Bang are “hidden gems” (high mean rating, low variance). Metropolitan hubs (HN, HCM) show high variance due to diverse visitor experiences

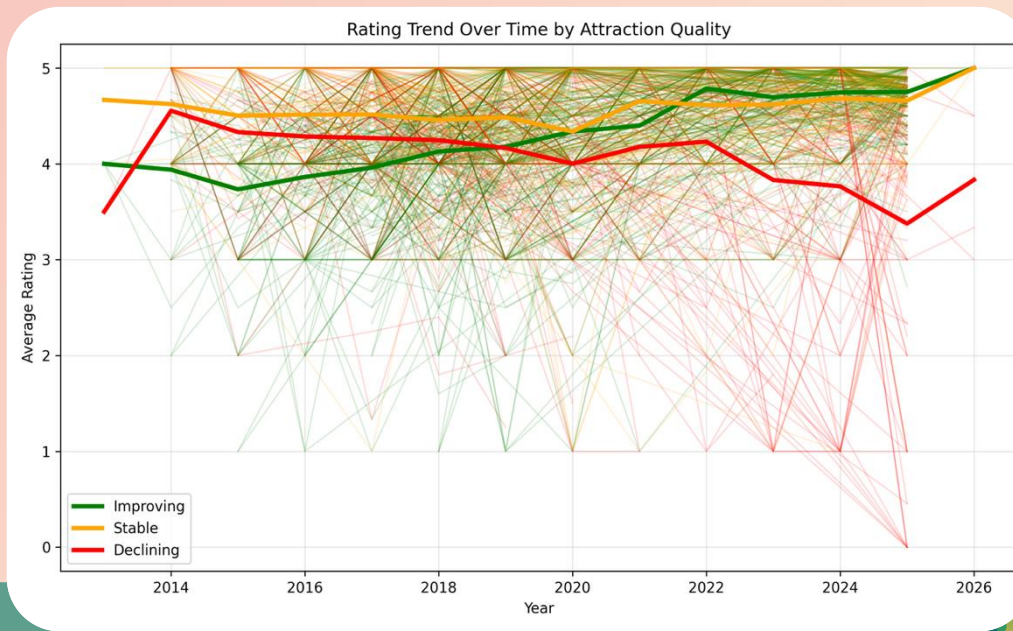
Rating trend over time

Rating trends computed using yearly average slope

Attractions categorized into:
Stable: ~60%, **Improving:** ~25%,
Declining: ~15%

Declining attractions show consistent rating decrease over multiple years

Temporal stability used as a quality reliability signal in TripMind



Methodology

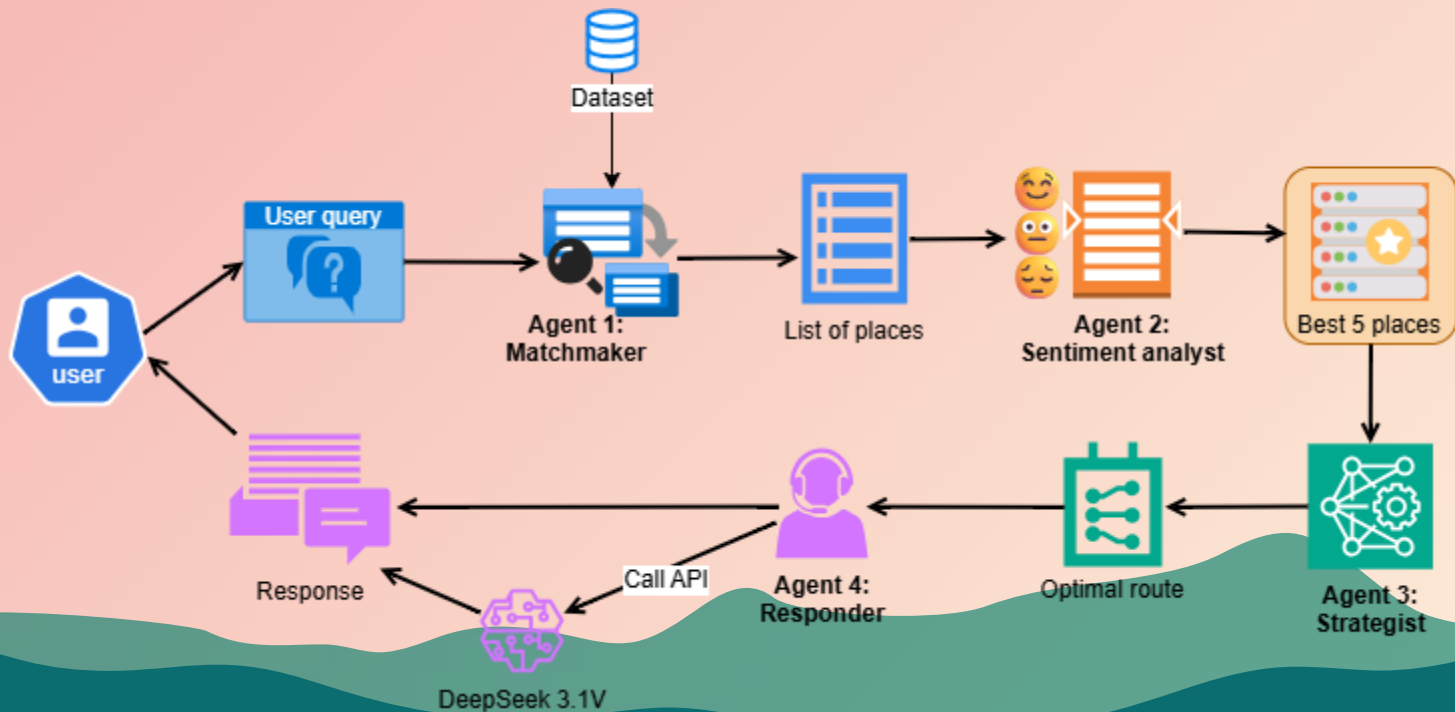


An illustration of a person with blue hair, wearing a yellow and orange hat, a white long-sleeved shirt, and red pants, carrying a large orange and black backpack. They are looking through blue binoculars. The background features a sunset with a large orange and red sun, white clouds, and a landscape with green hills, a blue body of water, and a traditional Chinese pagoda on a hill. A red flag with a yellow star is also visible on a hill.

Methodology

TripMind is a multi-agent system coordinated by a central Flask-based API, processing user queries through a sequential pipeline.

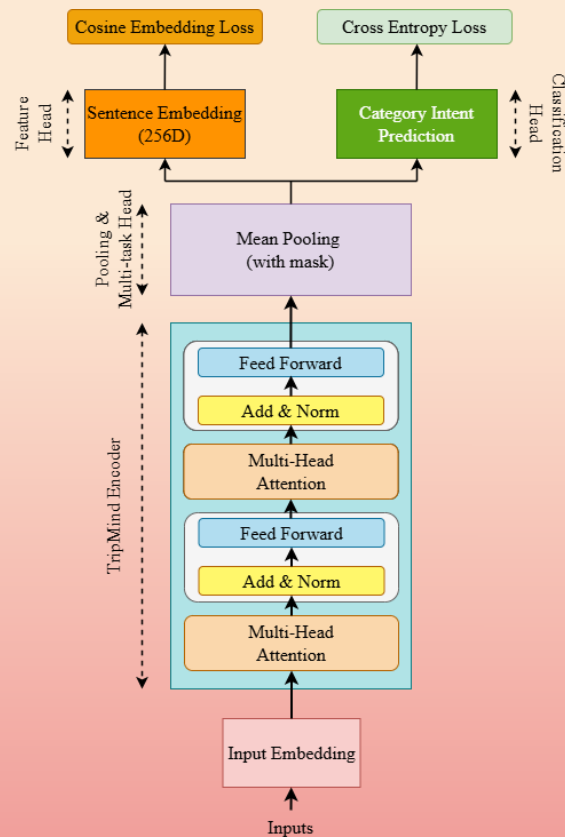
Overall architecture of TripMind



Agent 1

Role: To comprehend user intent and retrieve the most relevant candidates via semantic search.

Technology: A fine-tuned Transformer Encoder architecture.



How It Works ?



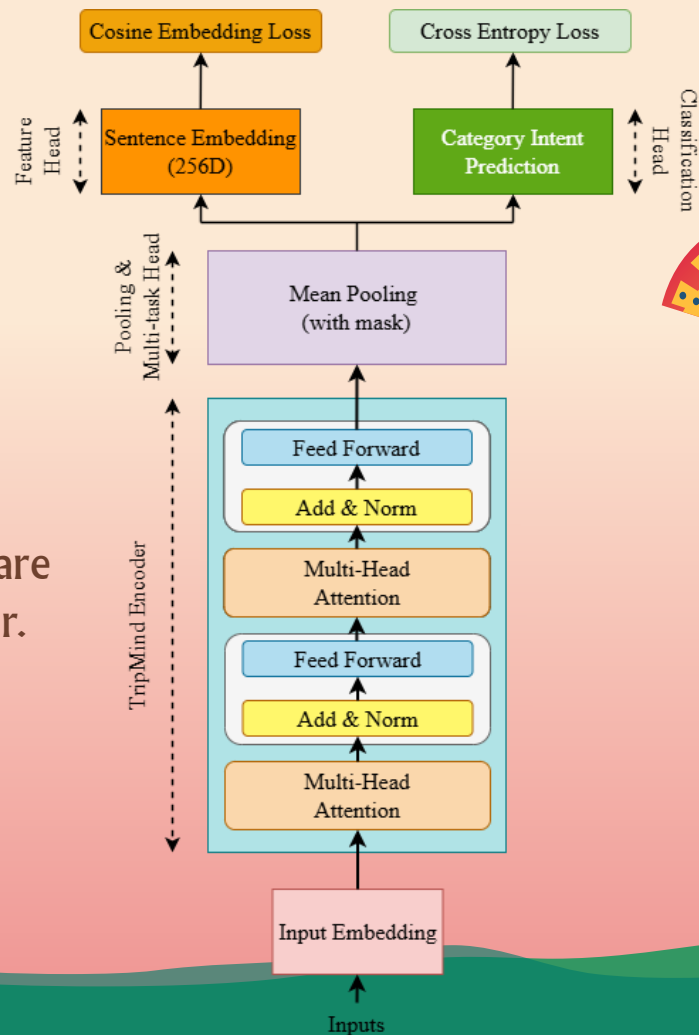
It converts your query and all location descriptions into rich numerical representations (256-dimensional vectors).



It then finds the locations whose vectors are semantically closest to your query's vector.



This is not just keyword matching; it's a search for contextual meaning.

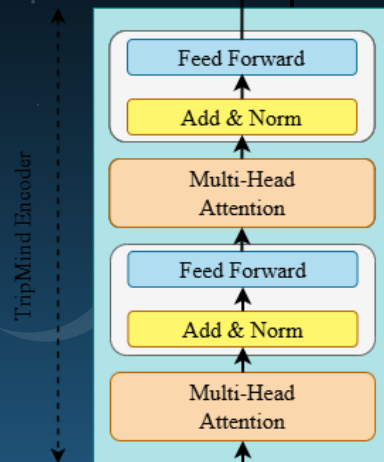


Transformer Encoder Layers

- Total 6 stacked Encoder layers
- Each layer consists of:
 - Multi-head self-attention (8 heads)
 - Position-wise feed-forward network
 - Hidden dimension:

$$4 \times d_{model} = 1024$$

- Uses `src_key_padding_mask` to ignore <PAD> tokens
→ Prevents attention noise in short texts



Transformer Encoder Layers

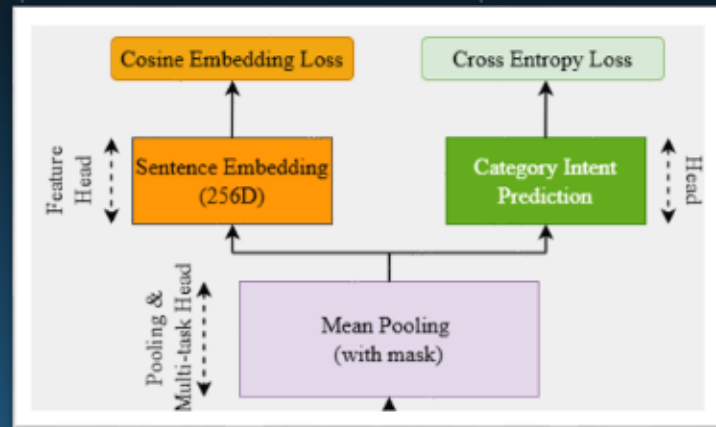
- **Masked Mean Pooling**
 - No [CLS] token is used
 - Sentence embedding is obtained via masked mean pooling:

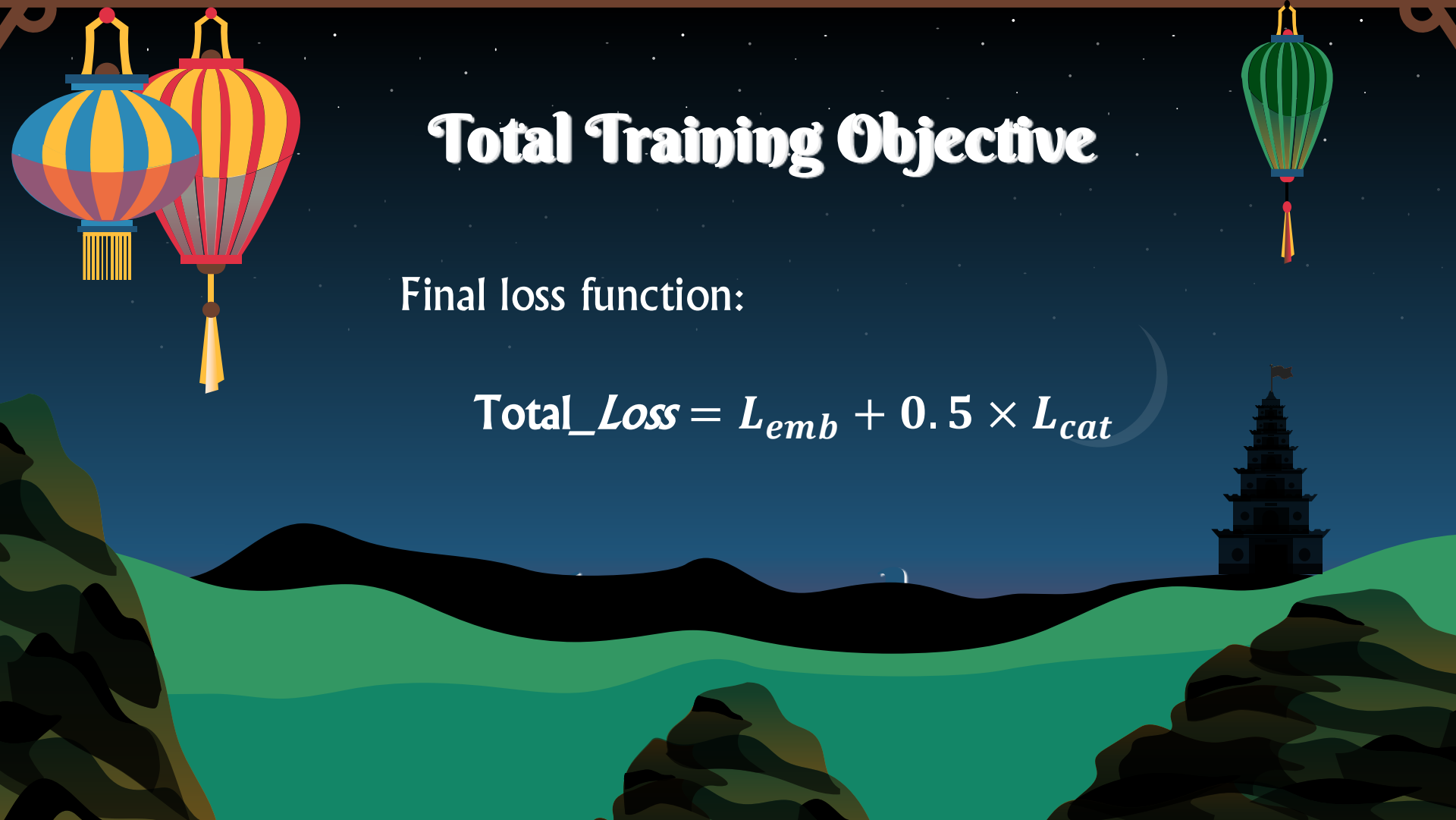
$$\text{Sentence_Emb} = \frac{\sum(\text{Output} \times \text{Mask})}{\sum \text{Mask}}$$

- **Multi-task Learning Objectives**
 - **Classification Loss:** Cross-Entropy Loss for intent / category prediction :

$$L_{cat} = - \sum y_i \log(\hat{y}_i)$$

- **Embedding Loss:** Cosine Embedding Loss (margin = 0.2)
 - Encourages:
 - Similar texts → closer vectors
 - Dissimilar texts → separated by margin

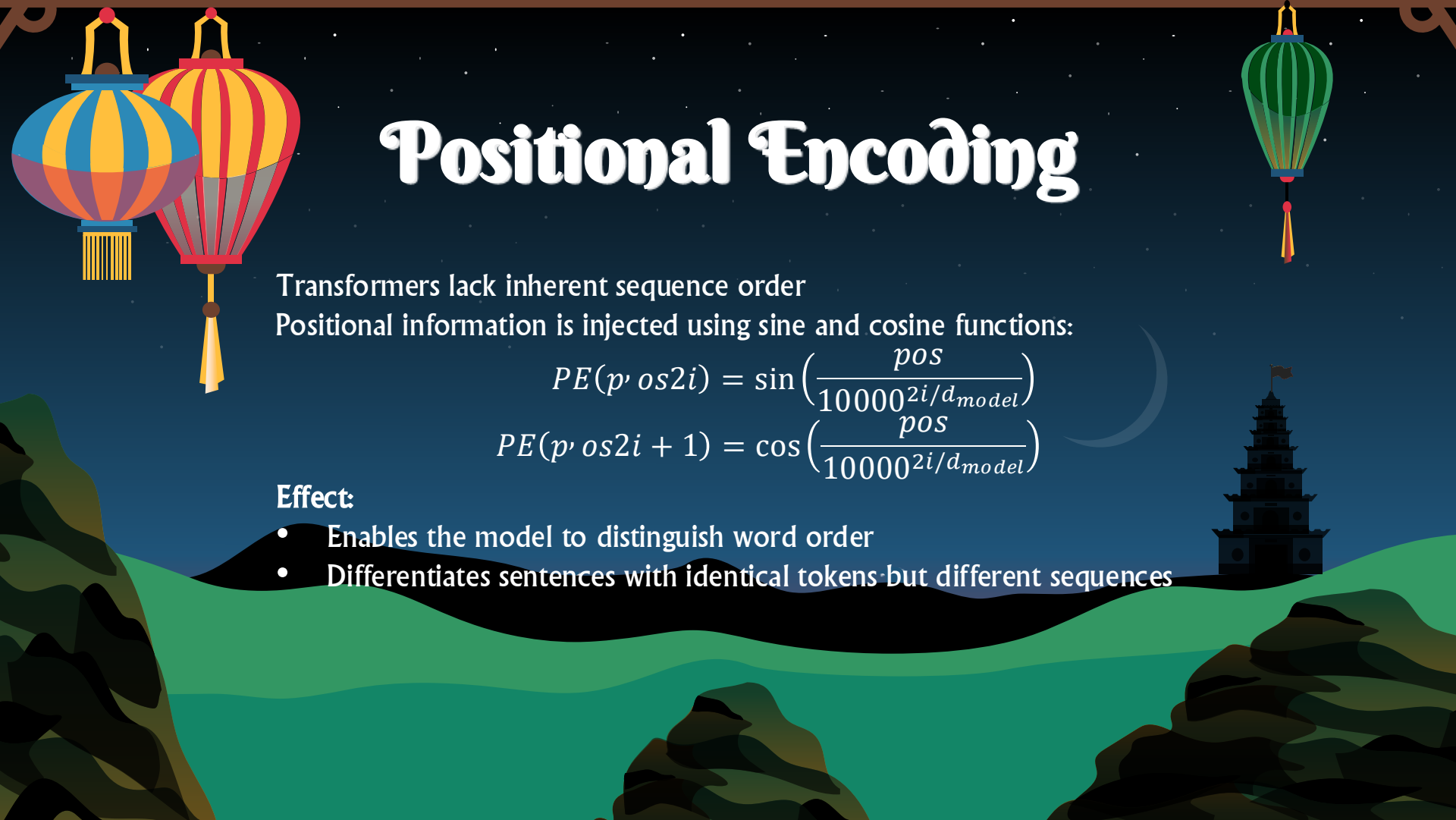


The background is a dark blue night sky with small white stars. In the upper left, two colorful lanterns (one blue and yellow, the other red and yellow) hang from the top. In the upper right, a green lantern hangs. In the lower right, a black pagoda with a red flag on top sits on a green hill. The foreground features dark green, jagged rock formations and a body of water in shades of green and blue. A thin, white crescent moon is visible in the sky.

Total Training Objective

Final loss function:

$$\text{Total_Loss} = L_{emb} + 0.5 \times L_{cat}$$



Positional Encoding

Transformers lack inherent sequence order

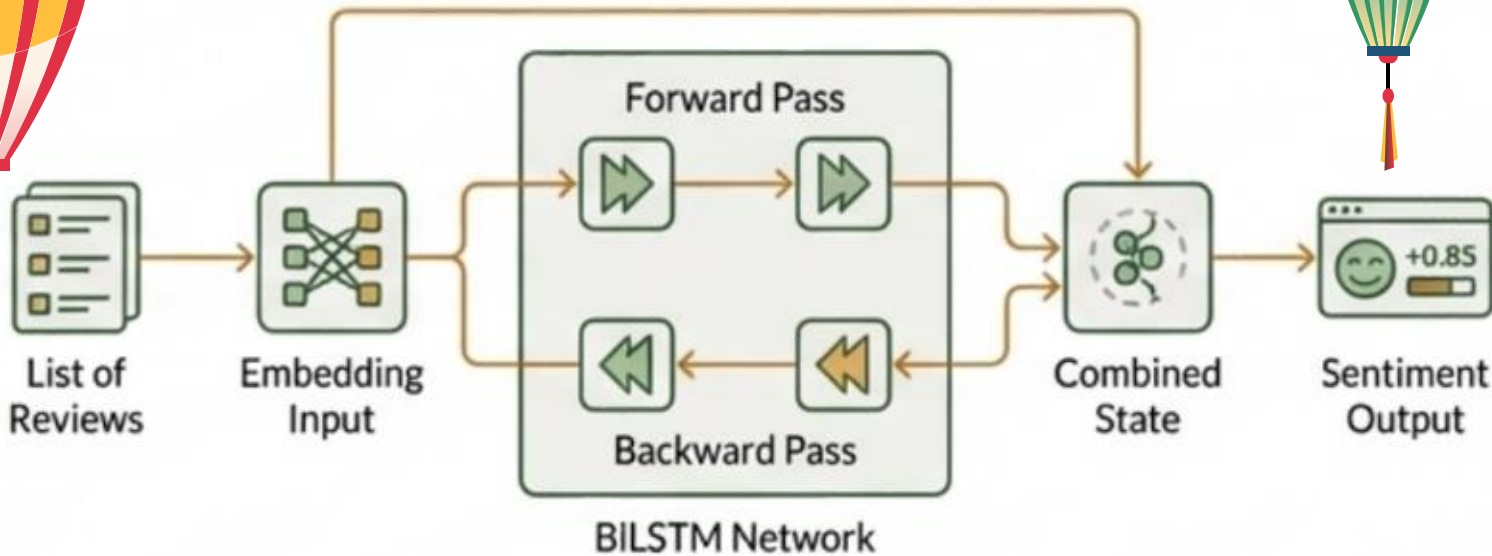
Positional information is injected using sine and cosine functions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Effect:

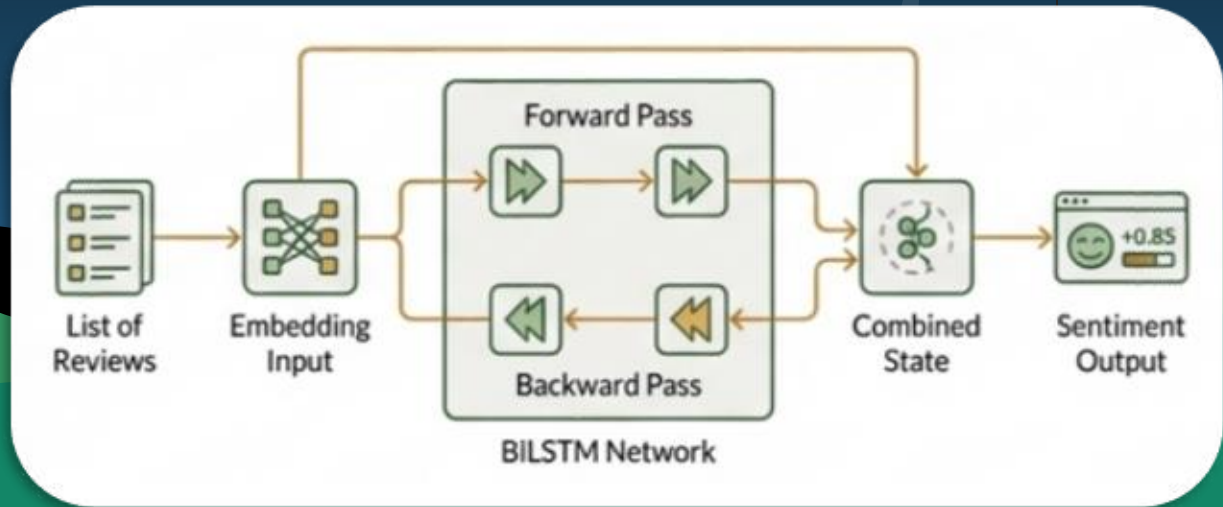
- Enables the model to distinguish word order
- Differentiates sentences with identical tokens but different sequences



Agent 2

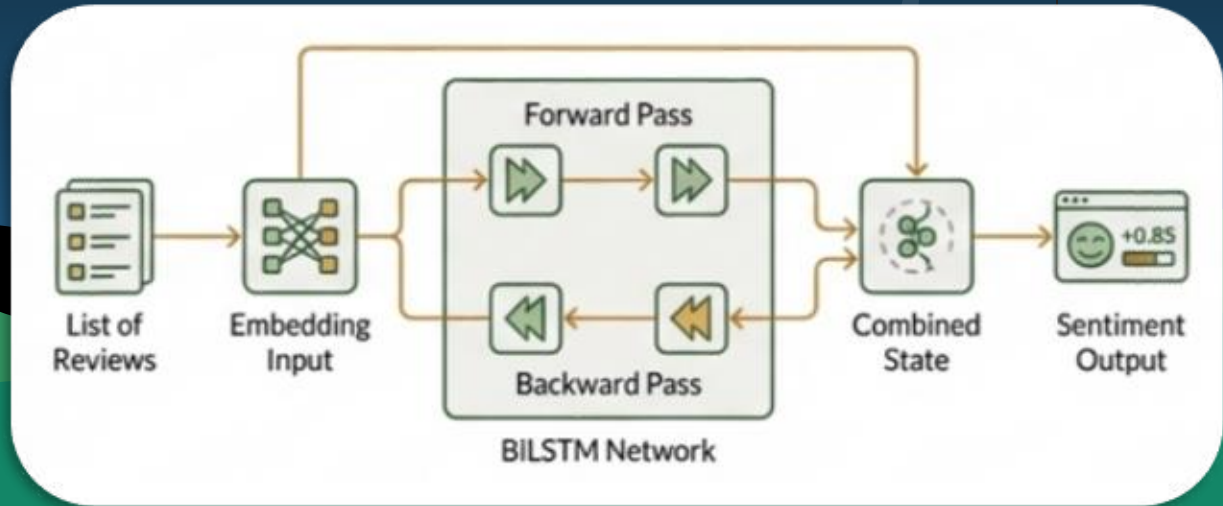
Sentiment Model Architecture

- The **embedding layer** maps each word into a dense vector of size 128
- We use one **Bidirectional LSTM layer** with hidden size 128
 - **Bidirectional processing** allows the model to read the sentence from left to right and right to left
 - Especially important for Vietnamese
(e.g., “không hề tệ” – *not bad at all*).
 - At each time step, forming a **256-dimensional representation**.



Sentiment Model Architecture

- **Mean pooling** is used to aggregate information, producing a fixed-length sentence for sentiment prediction.
- A **dropout layer** ($p = 0.3$) is applied to reduce overfitting

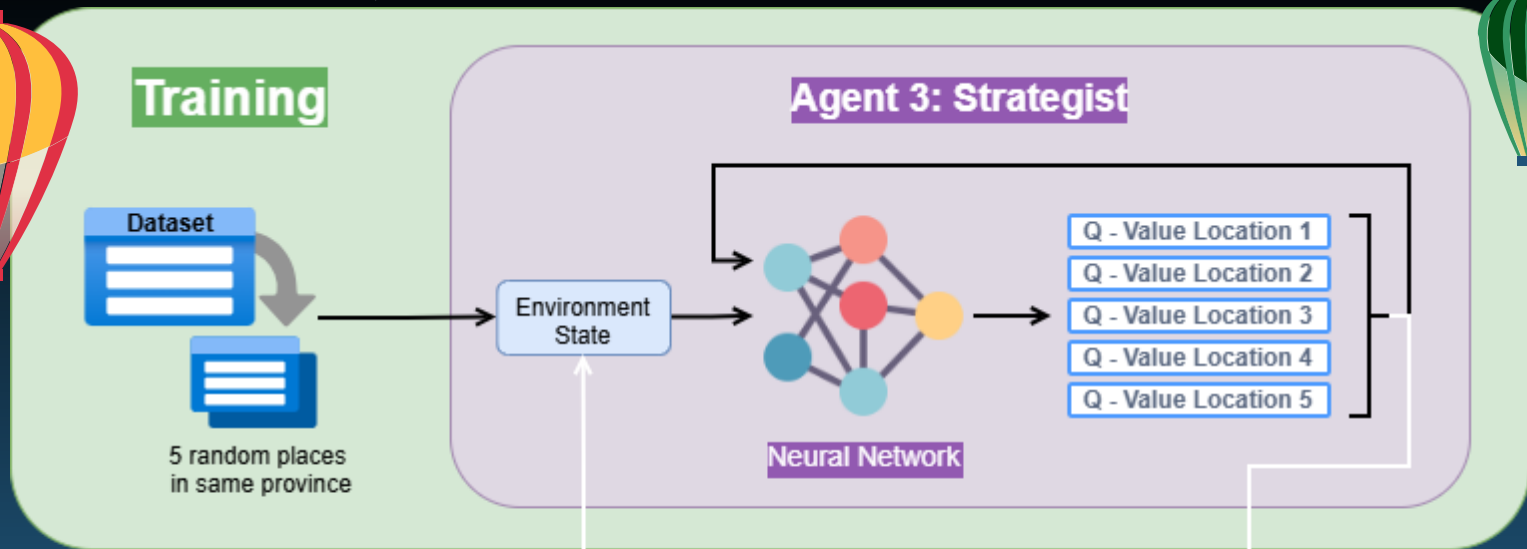


Loss Function

- **Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss)** for binary sentiment classification (positive vs. non-positive).
- This loss function integrates **sigmoid activation** and **cross-entropy** in a single step, improving **numerical stability** during training.

$$L_i = -[y_i \log \sigma(x_i) + (1 - y_i) \log(1 - \sigma(x_i))]$$

- Model parameters are optimized using the **Adam optimizer** with a learning rate of $\eta = 10^{-3}$, enabling fast and stable convergence.
- While **F1-score** is used for evaluation and threshold tuning, training is guided by BCE-based loss for reliable gradient optimization.



Best 5 places



Agent 2:
Sentiment analyst



Optimal Route

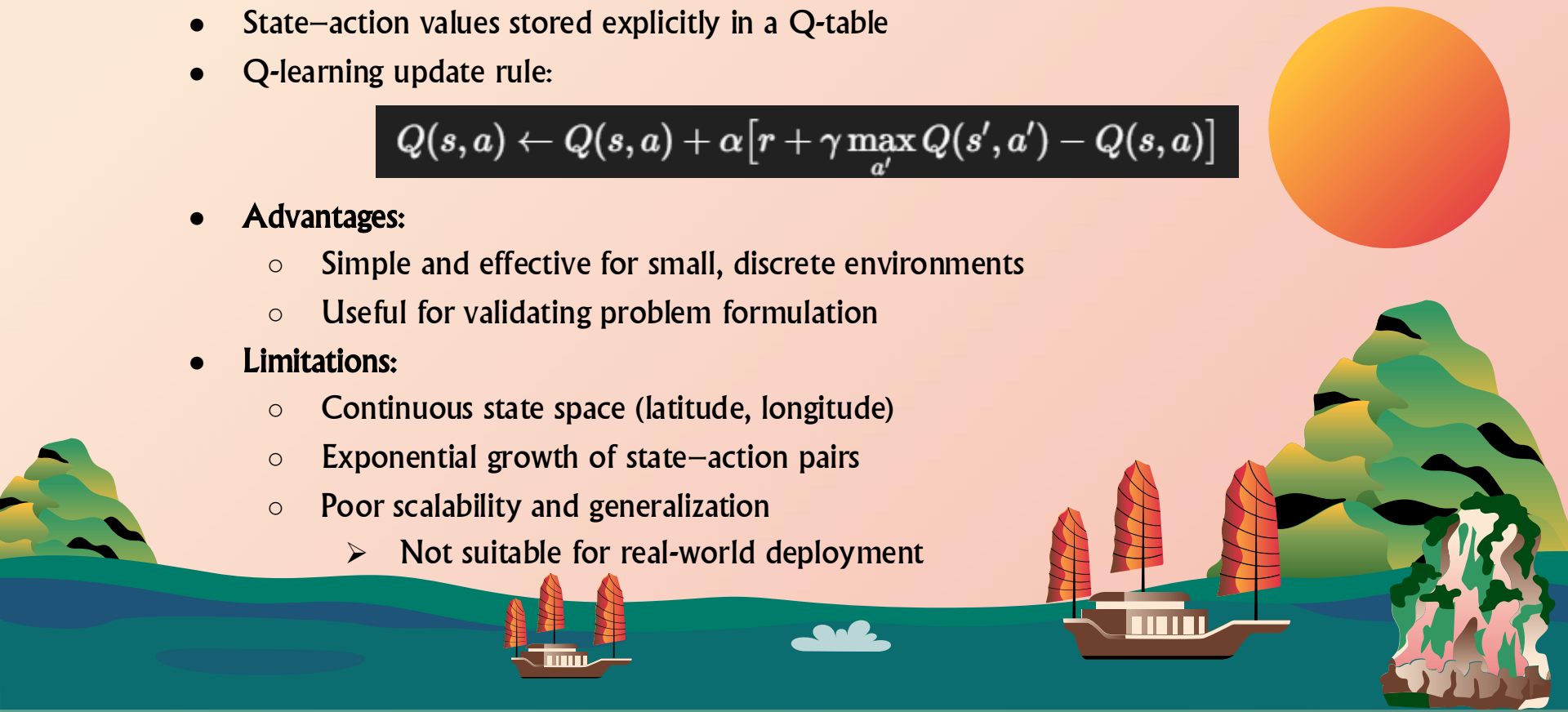
Agent 3

Initial Approach: Tabular Q-Learning

- State–action values stored explicitly in a Q-table
- Q-learning update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- **Advantages:**
 - Simple and effective for small, discrete environments
 - Useful for validating problem formulation
- **Limitations:**
 - Continuous state space (latitude, longitude)
 - Exponential growth of state–action pairs
 - Poor scalability and generalization
 - Not suitable for real-world deployment



Deep Q-Learning (DQN)

To overcome scalability issues, the system adopts **Deep Q-Learning**.
Q-function is approximated by a neural network:

$$Q(s, a; \theta)$$

Benefits:

- Handles continuous and high-dimensional states
- Learns generalized routing strategies
- Transfers knowledge across different location sets

Deep Q-Learning (DQN)

Training Objective

- The network is trained by minimizing the **Mean Squared Error (MSE)**:

$$L = \mathbb{E}[(Q(s, a) - (r + \gamma \max_{a'} Q'(s', a')))^2]$$

- Q' :target network

➤ Stabilizes training and improves convergence

Experiments



Agent 1: Training Setup

- **Purpose:** Foundational model for semantic retrieval and intent prediction
- **Training strategy:** Multi-task learning
 - Destination identification
 - Travel category classification

Key hyperparameters

- Epochs: 100, Batch size: 32
- Learning rate: 5×10^{-4} (AdamW)
- Encoder layers: 4, Attention heads: 8
- Vector dimension d_{model} **256** :
- Max sequence length: 100

Experimental environment

- **Training:** Kaggle + NVIDIA Tesla P100
- **Inference:** Apple M1 + MPS



Results

- **Loss curve:** Smooth exponential decay from $\sim 13.5 \rightarrow 0$
 - Indicates stable convergence and well-tuned learning rate
- **Accuracy curve:** Three learning phases
 1. Epochs 0–20: Initial adaptation (positional encoding)
 2. Epochs 20–60: Rapid semantic learning
 3. Epochs 80–100: Fine-tuning
- **Final training accuracy: 99.44%**
- **Final testing accuracy on destination: 46.90%**
- **Final testing accuracy on category: 77.51%**

➤ The model learns high-quality semantic representations, providing a strong foundation for downstream retrieval and ranking.

Results

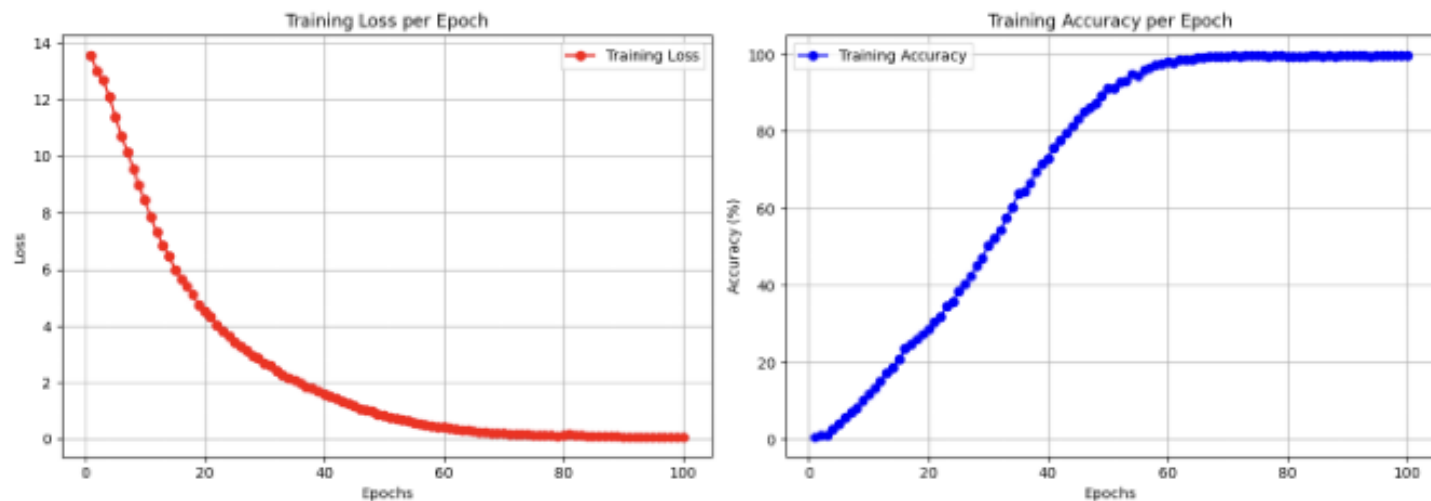
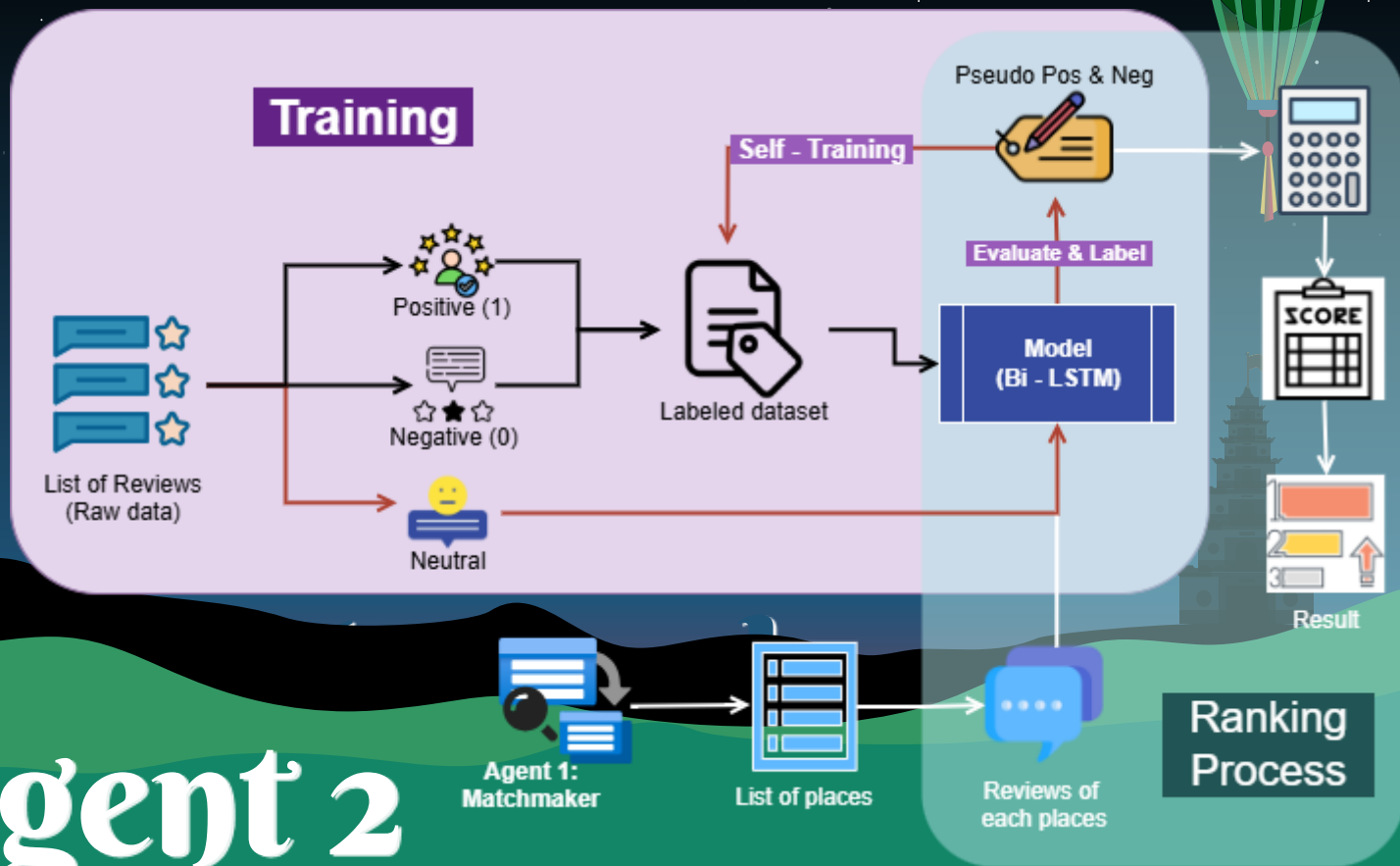


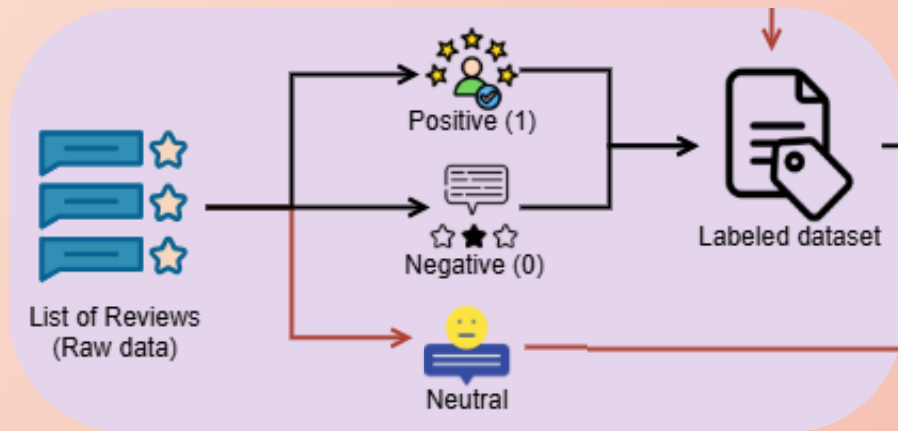
Figure 25: Training process of agent 1

Agent 2



Agent 2: Dataset & Preprocessing

- **Dataset:** User-generated reviews from social platforms
- **Each sample includes:** Review text, Star rating (1–5), Attraction ID
- **Labeling strategy**
 - ★ 1–2 stars → **Negative**
 - ★ 5 stars → **Positive**
 - ★ 3–4 stars → **Neutral**
(used later via self-training)
- **Class imbalance handling**
 - Positive samples are **downsampled** to match negative class
 - Remaining positives → **neutral pool** for self-training
- **Preprocessing:** Tokenization, Lowercasing, Noise removal



Early Stopping Strategy

- **Motivation**

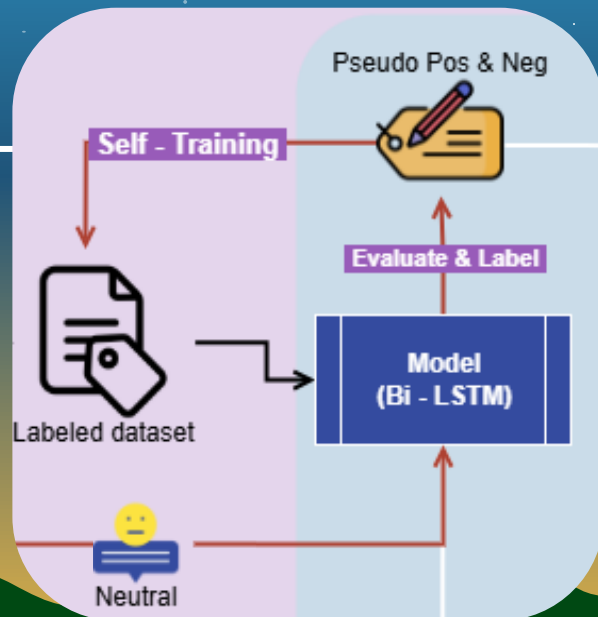
- Reviews are short, informal, and noisy
- Pseudo-labels may contain errors
- Training loss ↓ does **not always** mean performance ↑

- **Strategy**

- Monitor **validation loss**
- Apply early stopping with **patience**
- Stop training when validation loss stops improving

- **Benefits**

- Better generalization
- Stable training across initial & self-training phases
- Prevents performance degradation & avoid overfitting



Average Sentiment Score

$$S_{place} = \frac{1}{K} \sum_{i=1}^K P(\text{positive})_i$$

Captures the overall sentiment intensity of a location

Positive Review Ratio

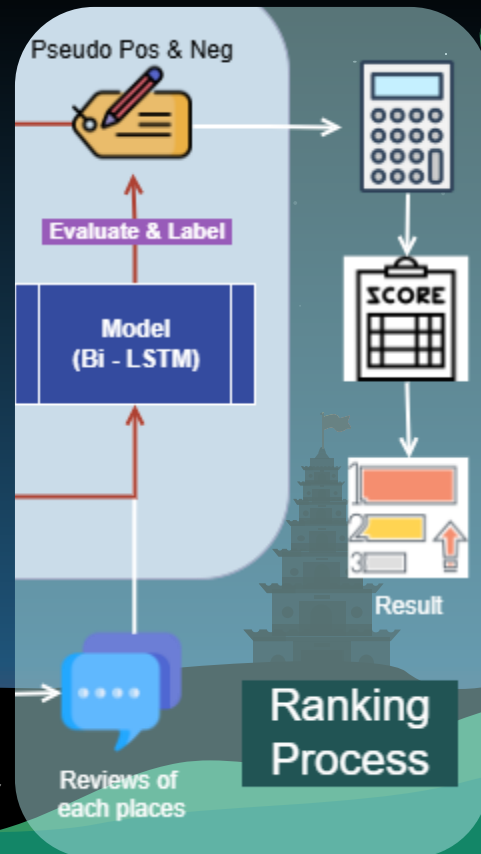
$$R_{pos} = \frac{1}{K} \sum_{i=1}^K \mathbb{I}(P(\text{positive})_i \geq \tau)$$

Measures the consistency of positive user feedback

Final Score

$$S_{final} = 0.7 \cdot S_{place} + 0.3 \cdot R_{pos}$$

Emphasizes overall sentiment while rewarding consistent positivity



→ Locations are ranked by S_{final} to get Top 5 locations

Evaluation Metrics

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

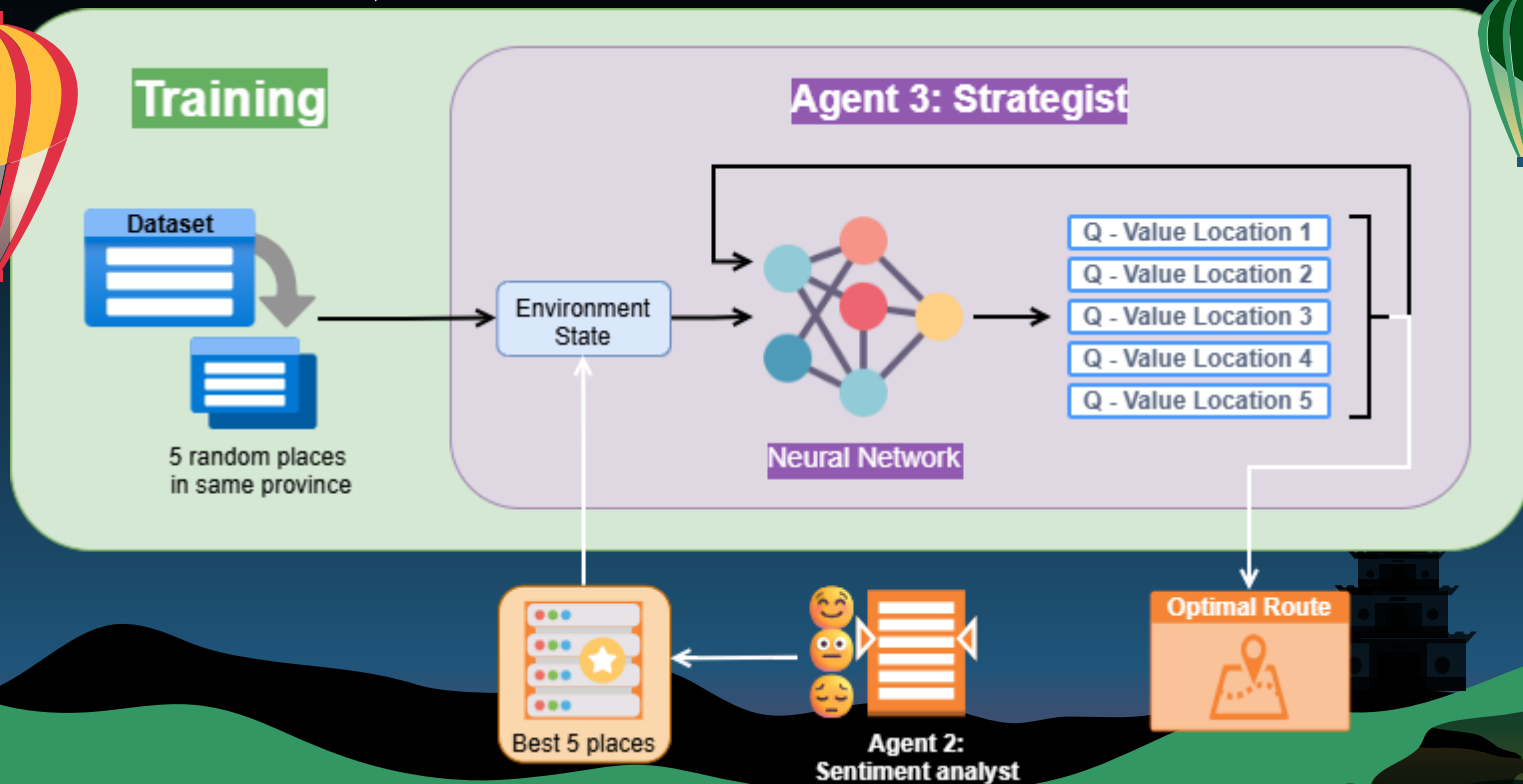
F1-score (primary metric)

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}$$

Why F1-score?

- Dataset is **imbalanced**
- Self-training introduces **pseudo-labeled noise**
- F1 balances **false positives & false negatives**
- More reliable than accuracy for sentiment classification



Agent 3

Agent 3 constructs an optimal visiting order for the **Top 5 selected locations**, closely related to the **TSP**, and is solved using **RL**.



Agent 3

- **State representation:** longitudes and latitudes of all 5 locations + visited status

$$s = [lat_1, lng_1, \dots, lat_5, lng_5, v_1, \dots, v_5]$$

- **Action space and action masking:**
 - At each timestep, the agent selects one of five discrete actions, each corresponding to visiting a specific location
 - Action masking: actions corresponding to already visited locations are masked out
 - Significantly improves training efficiency and convergence stability.



Agent 3

- **Deep Q-Network:**

- Input layer: 15-dimensional state vector (5 locations x 2 geographical coordinates + 5 binary elements of visited mask).
- Two fully connected hidden layers, each with 128 neurons and ReLU activation.
- Output layer: 5 neurons representing Q-values for each action.

- **Reward function:**

$$r_t = - distance(current_location, next_location)$$

→ Penalizes long moves and minimizes total route length

- **Training with multi-input episodes:**

- A province is selected randomly from dataset.
- Five locations from the same province are sampled.
- A new routing environment is instantiated.
- The agent interacts with the environment to collect experience.

Thanks for
your attention

